

Deepfake Detection Through Spectral-Spatial Fusion

Abstract: This report details a novel deep learning architecture designed for the robust detection of deepfake images. The proposed solution moves beyond traditional image-space analysis by adopting a dual-branch model that simultaneously processes spatial and frequency domain information. One branch, powered by a pre-trained EfficientNet, extracts high-level semantic features from the raw pixel data. The second branch analyzes the Discrete Cosine Transform (DCT) coefficients of the image, capturing subtle artifacts often introduced by generative models. These two complementary data streams are intelligently merged using a cross-attention fusion mechanism, allowing the model to weigh information from both domains dynamically. This spectral-spatial approach provides a more holistic view of the image, leading to a highly effective and generalizable deepfake classifier. The model's hyperparameters were systematically optimized using the Optuna framework to ensure peak performance.

1. Methodology

The overall approach involves preparing the data in two formats (image and DCT coefficients), feeding them into a specialized dual-branch neural network, and training the network to distinguish between real and fake images.

1.1. Dataset and Preprocessing

The model is trained on the CI-Fake dataset, which contains a collection of both real and synthetically generated images. The dataset is loaded and split into training (85%) and validation (15%) sets.

A critical preprocessing step involves handling potential data corruption. Each image file is verified to ensure its integrity before being included in the dataset; corrupted files are skipped. This step prevents training interruptions and ensures model stability.

1.2. Feature Engineering: The Role of the Discrete Cosine Transform (DCT)

The core innovation of this methodology is the use of frequency domain analysis alongside standard spatial analysis. Generative models, particularly those that involve up-sampling or are based on compression-de-compression cycles, often leave subtle periodic artifacts in the image. These artifacts are not easily visible to the human eye but can be revealed by transforming the image into its frequency components.

The Discrete Cosine Transform (DCT) is a technique that separates an image into different frequency bands. It is a fundamental component of the JPEG compression standard. The model computes the 2D DCT of the grayscale version of each image. To create a fixed-size feature vector, the top-left 16x16 block of DCT coefficients (representing the most significant low-frequency components) is selected and flattened into a 256-dimensional vector. This vector serves as the input to the frequency analysis branch of our model.

1.3. Data Augmentation

To improve model generalization and prevent overfitting, a series of data augmentations are applied to the training images. These transformations create modified versions of the training data, exposing the model to a wider variety of inputs. The augmentation pipeline includes:

- **Color Jitter:** Randomly alters the brightness, contrast, saturation, and hue of the image.
- **Random Affine:** Applies random rotations, translations, and shearing to the image.

These augmentations are applied only to the spatial (image) data, as the DCT features are intended to capture more fundamental, artifact-related patterns that should not be heavily distorted. Both training and validation images are resized to 256x256 and normalized using standard ImageNet statistics.

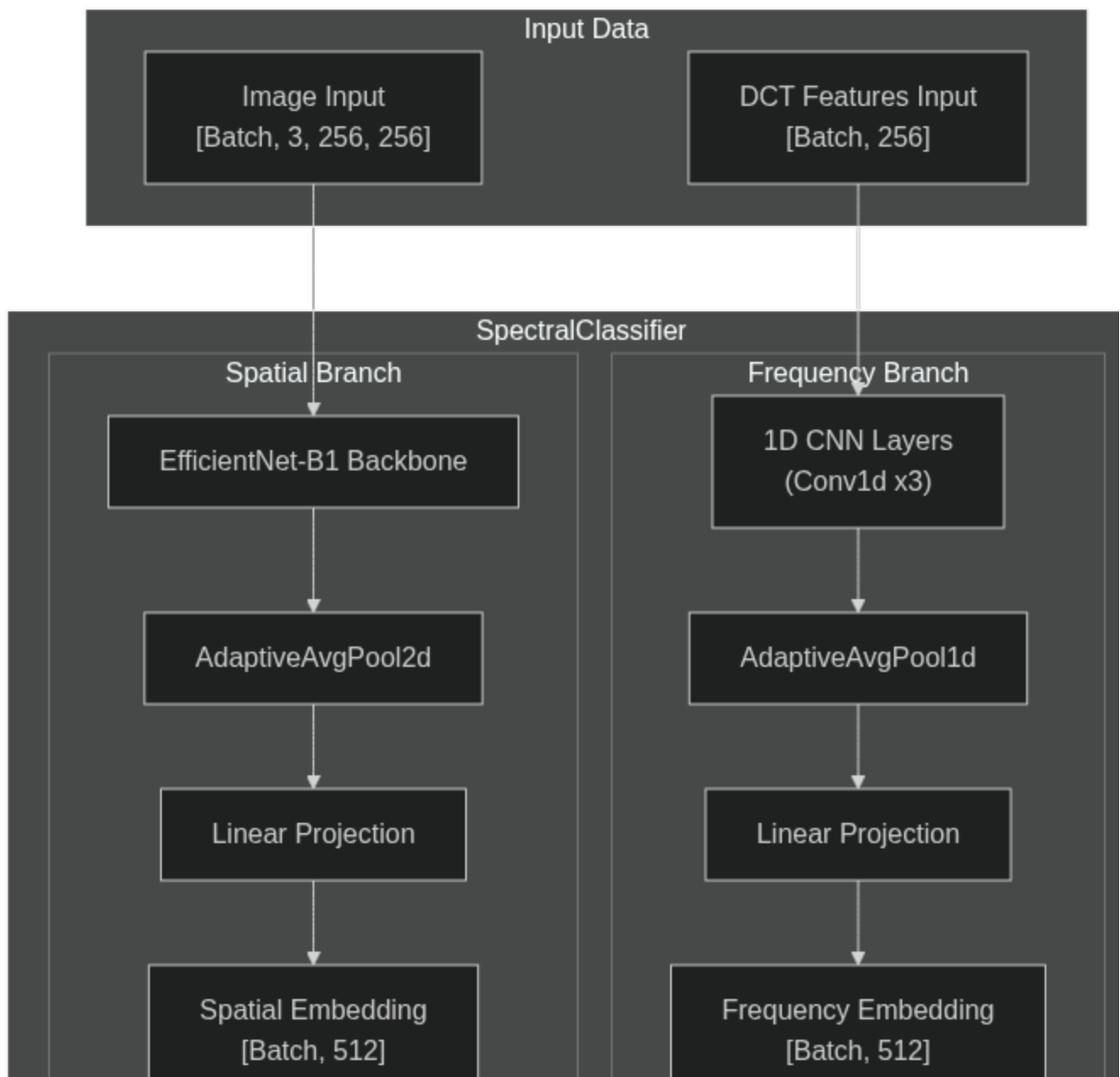
2. Model Architecture

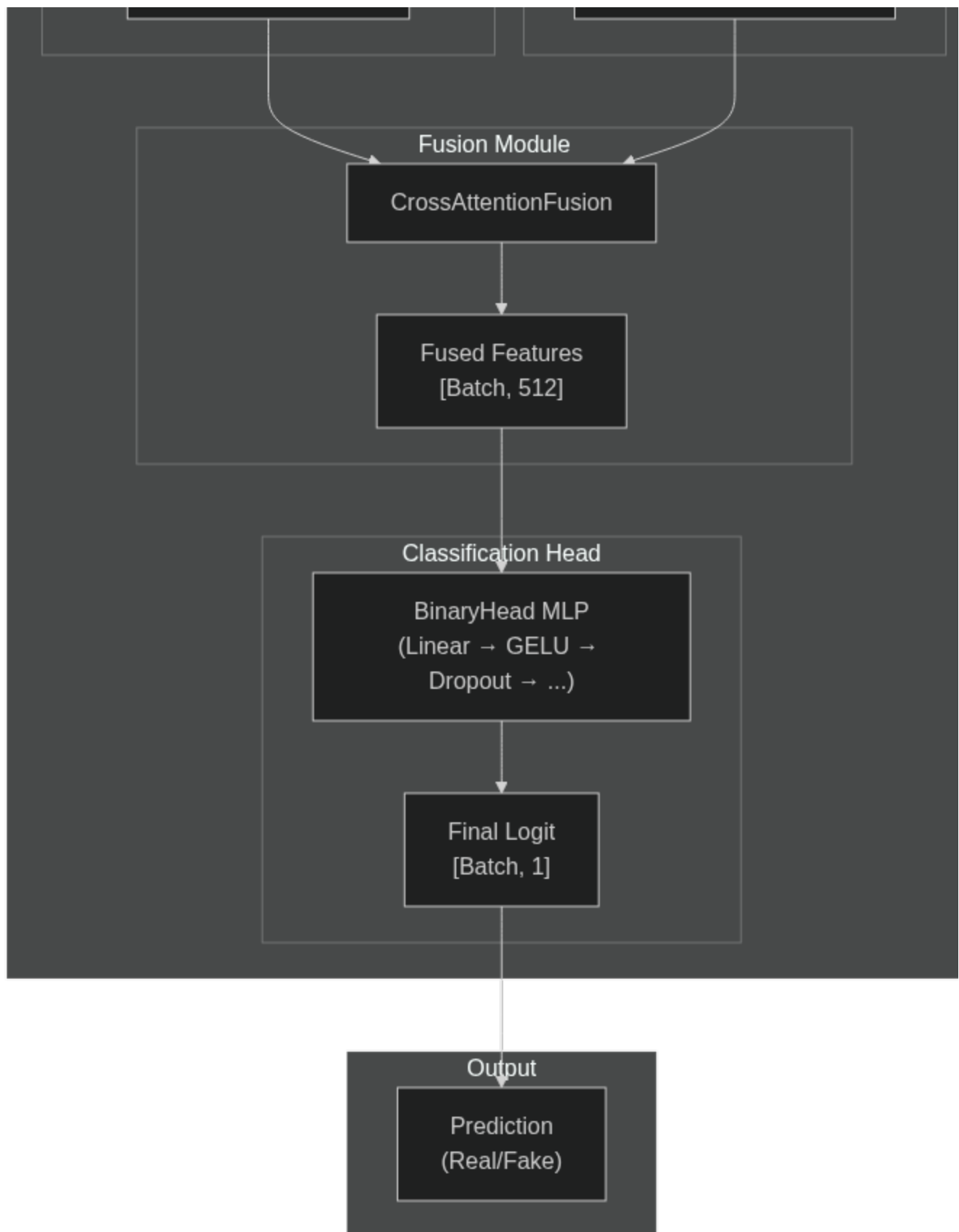
The SpectralClassifier is a multi-modal architecture composed of four primary

components: a Spatial Branch, a Frequency Branch, a Cross-Attention Fusion module, and a final Classification Head.

3.1. Overview of the Dual-Branch Approach

The model is designed as a two-branch network. This structure allows for the specialized processing of different data modalities—in this case, spatial pixel information and frequency-domain DCT coefficients. By learning from these complementary representations, the model can identify both high-level semantic inconsistencies (e.g., unnatural features) and low-level artifacts indicative of manipulation.





3.2. Spatial Branch: Leveraging Pre-trained CNNs

- **Purpose:** To extract high-level visual and semantic features from the image.
- **Implementation:** This branch utilizes EfficientNet-B1, a powerful and computationally efficient Convolutional Neural Network (CNN) pre-trained on the ImageNet dataset. EfficientNet is known for its excellent performance-to-size ratio, achieved through a balanced scaling of network depth, width, and resolution.
- **Mechanism:** The pre-trained backbone of EfficientNet-B1 processes the 256x256 input image. The final classification layer is removed (`nn.Identity()`), and the resulting feature map is passed through an adaptive average pooling layer. A fully connected layer then projects these features into a 512-dimensional embedding vector, which serves as the spatial representation of the image.

3.3. Frequency Branch: Learning from DCT Features

- **Purpose:** To analyze the 256-dimensional DCT feature vector and learn patterns indicative of digital manipulation.
- **Implementation:** This branch is a lightweight 1D Convolutional Neural Network. 1D CNNs are well-suited for processing sequential or signal-like data, making them a good fit for the flattened DCT coefficient vector.
- **Mechanism:** The input vector is treated as a 1D signal with a single channel. A series of three 1D convolutional layers with GELU activation functions progressively reduce the dimensionality of the signal while extracting relevant features. An adaptive average pooling layer and a final fully connected layer project these features into a 512-dimensional embedding vector, matching the dimensionality of the Spatial Branch output.

3.4. Cross-Attention Fusion: Combining Spatial and Frequency Insights

- **Purpose:** To intelligently merge the feature embeddings from the spatial and frequency branches. A simple concatenation would treat both feature sets as equally important. The cross-attention mechanism allows the model to dynamically weigh the features from one branch based on the context provided by the other.

- **Implementation:** A MultiheadAttention layer is the core of this module. This mechanism allows a model to jointly attend to information from different representation subspaces.
- **Mechanism:** The spatial embedding acts as the "query," while the frequency embedding provides the "key" and "value." This allows the model to ask, "Given these spatial features, which frequency features are most relevant?" The output of the attention mechanism is combined with the original spatial features and passed through a small multi-layer perceptron (MLP) to produce the final, fused 512-dimensional feature vector.

3.5. Classification Head

- **Purpose:** To take the fused feature vector and make a final binary prediction (real or fake).
- **Implementation:** A simple BinaryHead consisting of a sequence of linear layers with GELU activations and dropout for regularization.
- **Mechanism:** The 512-dimensional fused vector is passed through the MLP, which ultimately outputs a single logit. This logit represents the model's unnormalized prediction for the image being a deepfake.

4. Training and Hyperparameter Optimization

4.1. Training Regimen

The model is trained using a standard supervised learning procedure with the following components:

- **Loss Function:** BCEWithLogitsLoss, which combines a Sigmoid layer and Binary Cross-Entropy loss. This is numerically stable and appropriate for binary classification tasks.
- **Optimizer:** AdamW, an extension of the Adam optimizer that improves weight decay regularization.
- **Learning Rate Scheduler:** ReduceLROnPlateau. This scheduler monitors the validation loss and reduces the learning rate if it stops improving, allowing the model to fine-tune its weights more effectively.

4.2. Automated Hyperparameter Tuning with Optuna

To find the optimal combination of hyperparameters, the Optuna library is employed. Optuna is a hyperparameter optimization framework that automates the search for the best-performing settings. The following hyperparameters were tuned over 10 trials:

- **Batch Size:** 32, 64, 128
- **Learning Rate:** Log-uniform distribution between $1e-5$ and $1e-3$
- **Dropout Rate:** Float between 0.1 and 0.5

For each trial, a new model is instantiated and trained for 15 epochs. The validation accuracy is used as the metric to guide the optimization process, and the state of the best-performing model is saved. This systematic approach ensures that the final model is not just well-designed but also well-tuned for the specific task.